

Introduction to Machine Learning (25737-2)

Problem Set 4

Spring Semester 1403-04

Department of Electrical Engineering

Sharif University of Technology

Instructor: Dr. S. Amini

Due on , 1404 at 23:59



1 U-Net

In this question, we aim to explore the architectural features and training principles of U-Net, and discuss the Transposed Convolution operation. To answer the questions, read **paper** to understand the main ideas of the paper.

- (a) In this network, the encoder and decoder sections are connected through Skip Connections. Justify why these connections are used, based on your own understanding.
- (b) In training, the Random Deformation technique is used to increase the number of training samples. Try to simply explain how this technique is implemented and its effect on model performance.
- (c) Consider the following matrices. Using the specified filter and input, apply the Transposed Convolution operation.

$$\text{Input} = \begin{pmatrix} 3 & 2 \\ 1 & 4 \end{pmatrix} \quad \text{Filter} = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$$

Answer

- (a) Due to the Down Sampling performed in the encoder section, spatial fine details are lost. These connections allow precise and detailed information to be directly transferred to the decoder section for reconstruction.
These connections cause low-level and high-level feature maps to be directly combined, ultimately allowing image details such as edges to be reconstructed more accurately.
- (b) Due to the Down Sampling performed in the encoder section, spatial fine details are lost. These connections allow precise and detailed information to be directly transferred to the decoder section for reconstruction.
These connections cause low-level and high-level feature maps to be directly combined, ultimately allowing image details such as edges to be reconstructed more accurately.

(c) Considering the filter and input, we have:

stride = 1, padding = 0

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 4 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 2 & 4 \\ 0 & 6 & 8 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 3 & 6 & 0 \\ 9 & 12 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 4 & 8 \\ 0 & 12 & 16 \end{bmatrix} = \begin{bmatrix} 1 & 4 & 4 \\ 6 & 20 & 16 \\ 9 & 24 & 16 \end{bmatrix}$$

2 Backpropagation

The softmax function has the desirable property that it outputs a probability distribution, and is often used as activation function in many classification neural networks.

Consider a 2-layer neural network for K-class classification using softmax activation and cross-entropy loss, as defined below:

$$\begin{aligned} z^{[1]} &= W^{[1]}x + b^{[1]} \\ a^{[1]} &= \text{LeakyReLU}(z^{[1]}, \alpha = 0.01) \\ z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\ \hat{y} &= \text{softmax}(z^{[2]}) \\ L &= - \sum_{i=1}^K y_i \log(\hat{y}_i) \end{aligned}$$

where the model is given input x of shape $D_x \times 1$, and one-hot encoded label $y \in \{0, 1\}^K$. Assume that the hidden layer has D_a nodes, i.e., $z^{[1]}$ is a vector of size $D_a \times 1$. Recall the softmax function is computed as follows:

$$\hat{y} = \left[\frac{\exp(z_1^{[2]})}{Z}, \dots, \frac{\exp(z_K^{[2]})}{Z} \right]$$

where

$$Z = \sum_{j=1}^K \exp(z_j^{[2]})$$

- What are the shapes of $W^{[2]}$, $b^{[2]}$? If we were vectorizing across m examples, i.e., using a batch of samples $X \in \mathbb{R}^{D_x \times m}$ as input, what would be the shape of the output of the hidden layer?
- What is $\frac{\partial \hat{y}_k}{\partial z_k^{[2]}}$? Simplify your answer in terms of elements of $\hat{y}$.
- What is $\frac{\partial \hat{y}_k}{\partial z_i^{[2]}}$, for $i \neq k$? Simplify your answer in terms of element(s) of $\hat{y}$.
- Assume that the label y has 1 at its k^{th} entry, and 0 elsewhere. What is $\frac{\partial L}{\partial z_i^{[2]}}$? Simplify your answer in terms of $\hat{y}_i$. *Hint: Consider both cases where $i = k$ and $i \neq k$.*
- What is $\frac{\partial z^{[2]}}{\partial a^{[1]}}$? Refer to this result as δ_1 .
- What is $\frac{\partial a^{[1]}}{\partial z^{[1]}}$? Refer to this result as δ_2 . Feel free to use curly brace notation.
- Denote $\frac{\partial L}{\partial z^{[2]}}$ with δ_0 . What is $\frac{\partial L}{\partial W^{[1]}}$ and $\frac{\partial L}{\partial b^{[1]}}$? You can reuse notations from previous parts. *Hint: Be careful with the shapes.*

- (h) To avoid running into issues with numerical stability, one trick may be used when implementing the softmax function.

Let $m = \max_{i=1}^K z_i$ be the max of z_i , then

$$\hat{y}_i = \frac{\exp(z_i^{[2]} - m)}{\sum_{j=1}^K \exp(z_j^{[2]} - m)}$$

What is the numerical problem with the initial softmax computation? Why would the modified formula help resolve that problem?

Answer

- (a)

$$W^{[2]} \in \mathbb{R}^{K \times D_a}, \quad b^{[2]} \in \mathbb{R}^{K \times 1}.$$

The output has size $D_a \times m$.

- (b)

$$\frac{\partial \hat{y}_k}{\partial z_k^{[2]}} = \frac{\exp(z_k^{[2]})Z - \exp(z_k^{[2]})^2}{Z^2} = \hat{y}_k(1 - \hat{y}_k)$$

- (c)

$$\frac{\partial \hat{y}_k}{\partial z_i^{[2]}} = -\frac{\exp(z_k^{[2]}) \exp(z_i^{[2]})}{Z^2} = -\hat{y}_k \hat{y}_i$$

- (d)

$$L = -\log(\hat{y}_k)$$

When $i = k$,

$$\frac{\partial L}{\partial z_i^{[2]}} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_i^{[2]}} = -\frac{1}{\hat{y}_i} \hat{y}_i(1 - \hat{y}_i) = \hat{y}_i - 1$$

When $i \neq k$,

$$\frac{\partial L}{\partial z_i^{[2]}} = \frac{\partial L}{\partial \hat{y}_k} \frac{\partial \hat{y}_k}{\partial z_i^{[2]}} = \frac{1}{\hat{y}_k} \hat{y}_k \hat{y}_i = \hat{y}_i$$

- (e)

$$\delta_1 = W^{[2]}$$

- (f)

$$\delta_2 = \begin{cases} 1, & \text{if } z_i^{[1]} \geq 0 \\ 0.01, & \text{otherwise} \end{cases}$$

- (g) Using chain rule for partial derivatives, we get (note the element-wise product):

$$\frac{\partial L}{\partial W^{[1]}} = \delta_1^T * \delta_0 \circ \delta_2 * \mathbf{x}^T$$

$$\frac{\partial J}{\partial b^{[1]}} = \delta_1^T * \delta_0 \circ \delta_2$$

- (h) A big $z_i^{[2]}$ will lead to overflow. All very small $z_i^{[2]}$ will lead to division-by-zero error.

The modified formula ensures the maximum exponential result is 1, and the denominator is always greater than 1.

3 Vector Derivative

Consider the following functions:

$$f_1 \left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \right) = \begin{bmatrix} \frac{1}{\pi} \sin(\pi x_2) \\ e^{x_1} - x_2^2 \\ x_1 x_2 \end{bmatrix}$$

$$f_2 \left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \right) = \begin{bmatrix} x_1 + x_2 + x_3 \\ x_1^2 + x_2^2 + x_3^2 \end{bmatrix}$$

$$f(x) = (f_2 \circ f_1)(x)$$

Determine $\frac{\partial f}{\partial x}$ at point $x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ using only backpropagation.

Answer

For $x = (1, 2)^\top$ we first compute

$$f_1(x) = \begin{bmatrix} \frac{1}{\pi} \sin(\pi \cdot 2) \\ e^1 - 2^2 \\ 1 \cdot 2 \end{bmatrix} = \begin{bmatrix} 0 \\ e - 4 \\ 2 \end{bmatrix} =: y.$$

The Jacobian of $f_2(y) = [y_1 + y_2 + y_3, y_1^2 + y_2^2 + y_3^2]^\top$ is

$$J_{f_2}(y) = \begin{bmatrix} 1 & 1 & 1 \\ 2y_1 & 2y_2 & 2y_3 \end{bmatrix}, \quad \text{so at } y = (0, e - 4, 2)^\top : J_{f_2}(y) = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 2(e - 4) & 4 \end{bmatrix}.$$

For $f_1(x) = [\frac{1}{\pi} \sin(\pi x_2), e^{x_1} - x_2^2, x_1 x_2]^\top$ the Jacobian is

$$J_{f_1}(x) = \begin{bmatrix} 0 & \cos(\pi x_2) \\ e^{x_1} & -2x_2 \\ x_2 & x_1 \end{bmatrix}, \quad \text{so at } x = (1, 2)^\top : J_{f_1}(x) = \begin{bmatrix} 0 & 1 \\ e & -4 \\ 2 & 1 \end{bmatrix}.$$

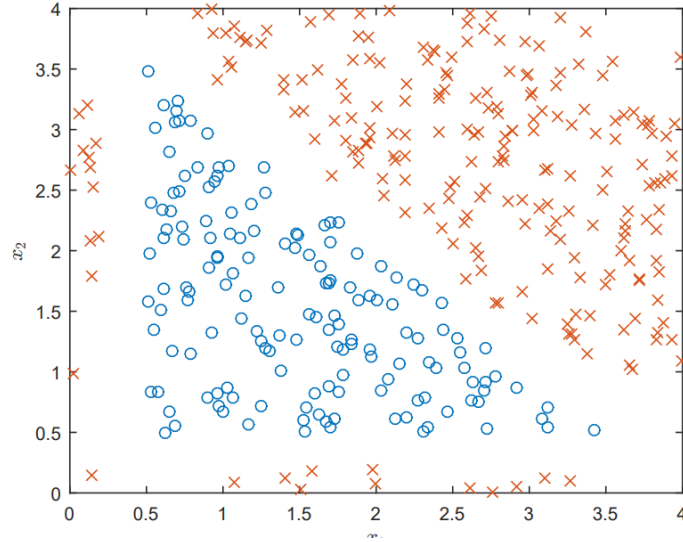
Using the chain rule for the composite map $f = f_2 \circ f_1$,

$$J_f(x) = J_{f_2}(f_1(x)) J_{f_1}(x) = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 2(e - 4) & 4 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ e & -4 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} e + 2 & -2 \\ 2e^2 - 8e + 8 & -8e + 36 \end{bmatrix}.$$

Hence $\partial f / \partial x$ at $x = (1, 2)$ equals the 2×2 matrix above.

4 A Simple Neural Network

Let $X = \{x^{(1)}, \dots, x^{(m)}\}$ be a dataset of m samples with 2 features, i.e. $x^{(i)} \in \mathbb{R}^2$. The samples are classified into 2 categories with labels $y^{(i)} \in \{0, 1\}$. A scatter plot of the dataset is shown below:



The examples in class 1 are marked as “x” and examples in class 0 are marked as “o”. We want to perform binary classification using a simple neural network with only one hidden layer.

Denote the two features x_1 and x_2 , the three neurons in the hidden layer h_1, h_2, h_3 , and the output neuron as o . Let the weight from x_i to h_j be $w_{i,j}^{[1]}$ for $i \in \{1, 2\}, j \in \{1, 2, 3\}$, and the weight from h_j to o be $w_j^{[2]}$. Finally, denote the intercept weight for h_j as $w_{0,j}^{[1]}$, and the intercept weight for o as $w_0^{[2]}$. For the loss function, we’ll use average squared loss instead of the usual negative log-likelihood:

$$l = \frac{1}{m} \sum_{i=1}^m (o^{(i)} - y^{(i)})^2,$$

where $o^{(i)}$ is the result of the output neuron for example i .

- Suppose that we use the sigmoid function as the activation function for h_1, h_2, h_3 and o . What is the gradient descent update to $w_{1,2}^{[1]}$, assuming we use a learning rate of α ? Your answer should be written in terms of $x^{(i)}, o^{(i)}, y^{(i)}$, and the weights.
- Now, suppose instead of using the sigmoid function for the activation function for h_1, h_2, h_3 and o , we instead used the step function $f(x)$, defined as

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

What is one set of weights that would allow the neural network to classify this dataset with 100% accuracy? Please specify a value for the weights in the order given below and explain your reasoning.

$$\begin{aligned} w_0^{[1],1} &= ?, & w_1^{[1],1} &= ?, & w_2^{[1],1} &= ? \\ w_0^{[1],2} &= ?, & w_1^{[1],2} &= ?, & w_2^{[1],2} &= ? \\ w_0^{[1],3} &= ?, & w_1^{[1],3} &= ?, & w_2^{[1],3} &= ? \\ w_0^{[2]} &= ?, & w_1^{[2]} &= ?, & w_2^{[2]} &= ?, & w_3^{[2]} &= ? \end{aligned}$$

Hint: There are three sides to a triangle, and there are three neurons in the hidden layer.

- (c) Let the activation functions for h_1, h_2, h_3 be the linear function $f(x) = x$ and the activation function for o be the same step function as before. Is there a specific set of weights that will make the loss 0? If yes, please explicitly state a value for every weight. If not, please explain your reasoning.

Answer

- (a) We seek the gradient descent update rule for the weight $w_{1,2}^{[1]}$ using a learning rate α , where the activation function for all neurons is the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$.

Forward pass equations (for a single sample i):

$$a_2^{(i)} = w_{0,2}^{[1]} + w_{1,2}^{[1]}x_1^{(i)} + w_{2,2}^{[1]}x_2^{(i)}, \quad h_2^{(i)} = \sigma(a_2^{(i)}), \quad a_o^{(i)} = w_0^{[2]} + w_1^{[2]}h_1^{(i)} + w_2^{[2]}h_2^{(i)} + w_3^{[2]}h_3^{(i)}, \quad o^{(i)} = \sigma(a_o^{(i)})$$

The mean squared error loss for a single example is:

$$l^{(i)} = (o^{(i)} - y^{(i)})^2$$

Gradient via the chain rule: We compute the derivative of the loss with respect to $w_{1,2}^{[1]}$ by applying the chain rule:

$$\frac{\partial l^{(i)}}{\partial w_{1,2}^{[1]}} = \frac{\partial l^{(i)}}{\partial o^{(i)}} \cdot \frac{\partial o^{(i)}}{\partial a_o^{(i)}} \cdot \frac{\partial a_o^{(i)}}{\partial h_2^{(i)}} \cdot \frac{\partial h_2^{(i)}}{\partial a_2^{(i)}} \cdot \frac{\partial a_2^{(i)}}{\partial w_{1,2}^{[1]}} = 2(o^{(i)} - y^{(i)}) \cdot o^{(i)}(1 - o^{(i)}) \cdot w_2^{[2]} \cdot h_2^{(i)}(1 - h_2^{(i)}) \cdot x_1^{(i)}$$

Gradient descent update:

$$w_{1,2}^{[1]} \leftarrow w_{1,2}^{[1]} - \alpha \cdot 2(o^{(i)} - y^{(i)}) o^{(i)}(1 - o^{(i)}) w_2^{[2]} h_2^{(i)}(1 - h_2^{(i)}) x_1^{(i)}$$

If using the average over all m samples:

$$w_{1,2}^{[1]} \leftarrow w_{1,2}^{[1]} - \frac{\alpha}{m} \sum_{i=1}^m 2(o^{(i)} - y^{(i)}) o^{(i)}(1 - o^{(i)}) w_2^{[2]} h_2^{(i)}(1 - h_2^{(i)}) x_1^{(i)}$$

- (b) We use a neural network with three hidden neurons to classify the points as shown in the provided chart. The network structure is as follows:

- Each hidden neuron corresponds to one edge of the triangle with vertices $(0.5, 0.5)$, $(3.5, 0.5)$, $(0.5, 3.5)$.
- All neurons use the step activation function:

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- The outputs of the three hidden neurons are combined by the output neuron to produce the final classification.

Edge equations:

$$\begin{aligned} \text{Edge 1: } x_2 &= 0.5 & \rightarrow h_1 &= f(x_2 - 0.5) \\ \text{Edge 2: } x_2 &= -x_1 + 4 & \rightarrow h_2 &= f(-x_1 - x_2 + 4) \\ \text{Edge 3: } x_1 &= 0.5 & \rightarrow h_3 &= f(x_1 - 0.5) \end{aligned}$$

Network equations:

$$\begin{cases} h_1 = f(x_2 - 0.5) \\ h_2 = f(-x_1 - x_2 + 4) \\ h_3 = f(x_1 - 0.5) \\ o = f(-h_1 - h_2 - h_3 + 2.5) \end{cases}$$

where o represents the final output.

Interpretation:

- If (x_1, x_2) is inside the triangle, then $h_1 = h_2 = h_3 = 1 \implies o = f(-3 + 2.5) = f(-0.5) = 0$.
- If (x_1, x_2) is outside the triangle, then at least one $h_i = 0$, so $-h_1 - h_2 - h_3 + 2.5 \geq -2 + 2.5 = 0.5 \implies o = f(0.5) = 1$.

Weights and biases:

$$\text{Neuron } h_1 : w_{1,1}^{[1]} = 0, w_{2,1}^{[1]} = 1, b_1^{[1]} = -0.5$$

$$\text{Neuron } h_2 : w_{1,2}^{[1]} = -1, w_{2,2}^{[1]} = -1, b_2^{[1]} = 4$$

$$\text{Neuron } h_3 : w_{1,3}^{[1]} = 1, w_{2,3}^{[1]} = 0, b_3^{[1]} = -0.5$$

$$\text{Output neuron } o : w_1^{[2]} = -1, w_2^{[2]} = -1, w_3^{[2]} = -1, b^{[2]} = 2.5$$

- (c) Let the activation functions for the hidden neurons h_1, h_2, h_3 be the linear function $f(x) = x$, and the output neuron use the binary threshold (step) function:

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Let the input be (x_1, x_2) . The hidden layer computes:

$$h_1 = w_{11}x_1 + w_{12}x_2 + b_1, h_2 = w_{21}x_1 + w_{22}x_2 + b_2, h_3 = w_{31}x_1 + w_{32}x_2 + b_3$$

The output neuron computes

$$o = f(v_1h_1 + v_2h_2 + v_3h_3 + b).$$

Substituting the expressions for h_1, h_2, h_3 , the network's output can be written as

$$o = f(ax_1 + bx_2 + c)$$

where a, b , and c are combinations of the above weights and biases. This is the form of a *single-layer perceptron*: it can only produce a linear decision boundary in the (x_1, x_2) plane.

However, the classification task assigns label 0 to points *inside* a triangle and label 1 to points outside. The region inside the triangle cannot be separated from the outside by a single straight line; it requires a more complex (piecewise linear) boundary. Therefore, it is impossible to find weights for this network that achieve zero classification loss in this task.

5 Image Classification and CNNs

Two historians approach you for your deep learning expertise. They want to classify images of historical objects into 3 classes depending on the time they were created:

- Antiquity ($y = 0$)
 - Middle Ages ($y = 1$)
 - Modern Era ($y = 2$)
- (a) Over the last few years, the historians have collected nearly 5,000 hand-labelled RGB images. Before training your model, you want to decide the image resolution to be used. Why is the choice of image resolution important?
- (b) You have now figured out a good image resolution to use. After visually inspecting the dataset, you realize that the training set only contains pictures taken during the day, whereas the dev (validation) set only has pictures taken at night. Explain what is the issue and how you would correct it.
- (c) As you train your model, you realize that you do not have enough data. Cite 3 data augmentation techniques that can be used to overcome the shortage of data.
- (d) You come up with a CNN classifier. For each layer, calculate the number of weights, number of biases and the size of the associated feature maps.

The notation follows the convention:

- CONV- K - N denotes a convolutional layer with N filters, each of size $K \times K$, padding and stride parameters are always 0 and 1 respectively.
- POOL- K indicates a $K \times K$ pooling layer with stride K and padding 0.
- FC- N stands for a fully-connected layer with N neurons.

Layer	Activation map dimensions	Number of weights	Number of biases
INPUT	$128 \times 128 \times 3$	0	0
CONV-9-32			
POOL-2			
CONV-5-64			
POOL-2			
CONV-5-64			
POOL-2			
FC-3			

- (e) Why is it important to place non-linearities between the layers of neural networks?
- (f) Following the last FC-3 layer of your network, what activation must be applied? Given a vector $a = [0.3, 0.3, 0.3]$, what is the result of using your activation on this vector?
- (g) You find online that the exact same network has already been trained on 1,000,000 historical objects from a slightly different time period. What is the name of the method that could reuse these pretrained weights for the task at hand? What are the new hyperparameters to tune for this method?

Answer

- (a) There is a trade-off between accuracy and model complexity when choosing the image resolution. Higher resolutions allow the model to capture more detailed features from the images, which can potentially improve accuracy. However, they also increase the input size, leading to a larger and more complex model with more parameters. This makes the model computationally heavier, requires more memory and processing power, and increases the risk of overfitting—especially when the dataset is small.

- (b) This situation creates a domain mismatch because the training and dev sets come from different distributions (day vs. night). As a result, hyperparameter tuning on the dev set may not reflect true performance, leading to poor generalization on new data.

Solution: Randomly mix day and night images in both training and dev sets, then resplit, so each set represents the full range of conditions.

- (c)
- **Random Rotation:** Randomly rotate images by small angles to help the model learn rotational invariance.
 - **Horizontal Flipping:** Flip images horizontally to effectively double the dataset and help the model generalize to mirrored objects.
 - **Random Cropping or Scaling:** Randomly crop or resize images to simulate zooming in and out, increasing robustness to different object sizes and viewpoints.

Any other correct data augmentation technique is also accepted.

(d)

Layer	Activation map dimensions	Number of weights	Number of biases
INPUT	$128 \times 128 \times 3$	0	0
CONV-9-32	$120 \times 120 \times 32$	$32 \times (9 \times 9 \times 3)$	32
POOL-2	$60 \times 60 \times 32$	0	0
CONV-5-64	$56 \times 56 \times 64$	$64 \times (5 \times 5 \times 32)$	64
POOL-2	$28 \times 28 \times 64$	0	0
CONV-5-64	$24 \times 24 \times 64$	$64 \times (5 \times 5 \times 64)$	64
POOL-2	$12 \times 12 \times 64$	0	0
FC-3	3	$3 \times (12 \times 12 \times 64)$	3

- (e) Non-linearity introduces more degrees of freedom to the model. It lets it capture more complex representations which can be used towards the task at hand. A deep neural network without non-linearities is essentially a linear regression.
- (f) Softmax is the one that is used as it can output class probabilities. Output is $[0.33, 0.33, 0.33]$.
- (g) Transfer learning.

How many layers of the pretrained model to freeze, how many extra layers to add, and how many of the weights in the new network we want to initialize with the pretrained weights.

6 Imbalance, Regularization, and Backpropagation

You're trying to classify RGB images in giraffe present (1) and giraffe absent (0) using a deep neural network. Unfortunately, your data set is imbalanced. The class counts are:

- 2000 images with a giraffe
 - 200 examples with no giraffe
- (a) Name two data augmentation techniques you could use to help address the class imbalance problem and give a brief explanation about them.

Instead of data augmentation, you want to experiment with other techniques. Here's the architecture of your network:

$$\begin{aligned}
z_1 &= W_1 x^{(i)} + b_1 \\
a_1 &= \text{ReLU}(z_1) \\
z_2 &= W_2 a_1 + b_2 \\
\hat{y}^{(i)} &= \sigma(z_2) \\
L^{(i)} &= \alpha \cdot y^{(i)} \cdot \log(\hat{y}^{(i)}) + \beta \cdot (1 - y^{(i)}) \cdot \log(1 - \hat{y}^{(i)}) \\
J &= -\frac{1}{m} \sum_{i=1}^m L^{(i)}
\end{aligned}$$

The dimensions are as follows: $\hat{y}^{(i)} \in \mathbb{R}$, $y^{(i)} \in \mathbb{R}$, $x^{(i)} \in \mathbb{R}^{D_x \times 1}$, $W_1 \in \mathbb{R}^{D_{a1} \times D_x}$, $W_2 \in \mathbb{R}^{1 \times D_{a1}}$. Note m is the size of the dataset and that the RGB images are flattened into vectors of length D_x before being fed into the network.

- (b) What are the dimensions of b_1 and b_2 ?
- (c) Let's focus on the hyperparameters α and β of the loss function.
 - (i) Why are α and β useful?
 - (ii) What is a reasonable pair of values for (α, β) ? Provide specific values for these weightings.

(d) Backpropagation

In this part (d), feel free to use your results from earlier questions.

- (i) What is $\frac{\partial J}{\partial \hat{y}}$?
- (ii) What is $\frac{\partial \hat{y}^{(i)}}{\partial z_2}$?
- (iii) What is $\frac{\partial z_2}{\partial a_1}$?
- (iv) What is $\frac{\partial a_1}{\partial z_1}$?
- (v) What is $\frac{\partial z_1}{\partial W_1}$?
- (vi) What is $\frac{\partial J}{\partial W_1}$?

(e) Normalization

- (i) You decide to add L2 regularization to this model. Write your new cost function. Assume that the value of any regularization constant(s) is 1.
- (ii) Using this new cost function, write down the update rule for W_1 .
Hint: You should reuse some of your work from previous parts. Assume you are using gradient descent without optimizers. Use η as your learning rate.
- (iii) Suppose you used L1 regularization instead. How would you expect the weights learned using L1 regularization to differ from those learned using L2 regularization?

Answer

- (a) (i) Gaussian Blur:

Apply Gaussian blur to images of the minority “giraffe absent” class. This technique adds variety by simulating out-of-focus visuals, which helps the model generalize better by exposing it to a range of image sharpness.

- (ii) Translation:

Translate (shift) the minority class images horizontally or vertically by a few pixels. This creates new, slightly altered versions of the original images, increasing diversity in the minority class and helping the network become robust to object positions.

- (iii) Reflection (Horizontal Flip):

Reflect the minority class images across the vertical axis (horizontal flipping). This produces mirrored versions of the original images, further increasing the number of training examples for the underrepresented class.

Any other correct data augmentation technique is also accepted.

- (b)

$$b_1 \in \mathbb{R}^{D_{a_1} \times 1}$$

$$b_2 \in \mathbb{R}^{1 \times 1}$$

- (c) (i) Weighting how much each class contributes to the loss function can help gradient descent because the network will take larger steps when learning from instances of the underrepresented class.

- (ii) $\alpha = 0.1$, $\beta = 1$. Roughly, the ratio should be somewhere near $\beta = 10 * \alpha$ but not ridiculously large or small.

- (d) (i)

$$-\frac{1}{m} \sum_i \delta_1^{(i)}$$

where

$$\delta_1^{(i)} = \alpha * \frac{y^{(i)}}{\hat{y}^{(i)}} - \beta * \frac{(1 - y^{(i)})}{1 - \hat{y}^{(i)}}$$

- (ii)

$$\delta_2 = \sigma(z_2)(1 - \sigma(z_2))$$

- (iii)

$$\delta_3 = W_2$$

- (iv)

$$\delta_4 = \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

- (v)

$$x^{(i)T}$$

- (vi) There's multiple ways to express this. An example:

$$\delta_5 = -\frac{1}{m} \sum_i \delta_1^{(i)} * \delta_2 * (\delta_3 \circ \delta_4) * \delta_5$$

Where $\delta_1^{(i)}$, δ_2 are scalars, $\delta_3, \delta_4 \in \mathbb{R}^{D_{a_1} \times 1}$, $\delta_5 \in \mathbb{R}^{1 \times D_x}$

(e) (i)

$$J = - \sum_i (\alpha * (1 - y^{(i)}) * \log(1 - \hat{y}^{(i)}) + \beta * y^{(i)} * \log(\hat{y}^{(i)})) + \|W_2\|_2^2 + \|W_1\|_2^2$$

(ii)

$$W_1' = W_1 - \eta * \left(\frac{\partial J}{\partial W_1} + 2 * W_1 \right)$$

(iii) L1 weights will probably be more sparse.